

记忆感知的大模型 KVCache 优化

以 KVCache 为中心的开源推理框架 Mooncake 的技术实践

马腾 | kvcache.ai / Mooncake 开源社区
AICon 上海 2026

AICon
全球人工智能开发与应用大会

目录

- 01 背景：从对话到多智能体协作
- 02 核心视角：KVCache 即物理工作记忆
- 03 Mooncake: KVCache-Centric 分离架构
- 04 实战: OpenClaw + Mooncake 尾延迟优化
- 05 多 Agent 记忆优化的关键技术
- 06 协议与缓存的融合 — 未来展望与社区路线

01

背景：从对话到多智能体协作

单一对话 → 工具调用 → 多 Agent 协作

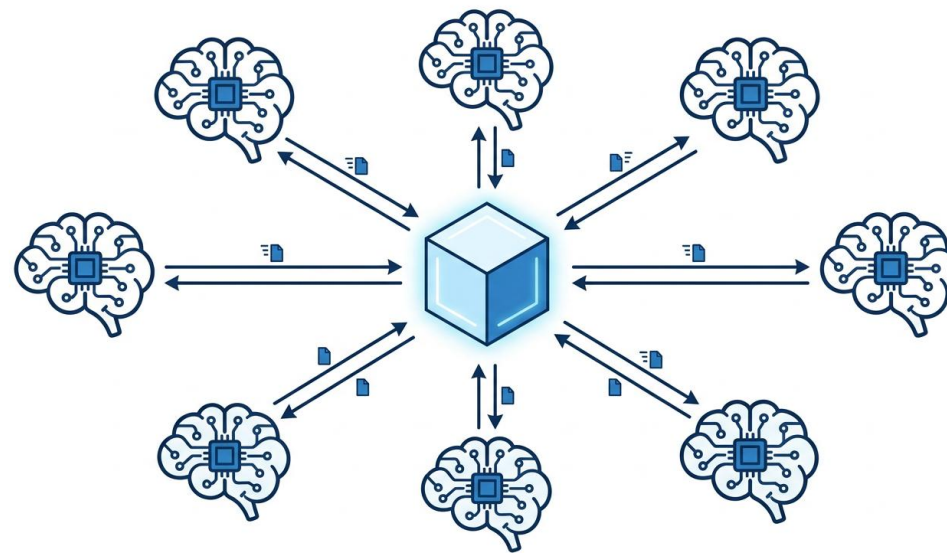
Multi-Agent 时代的到来

单一对话 → 工具调用 → 多 Agent 协作

系统提示词 + 工具描述 + 多轮历史 + Agent 间状态

= 庞大的"工作记忆"

从简单的人机对话，到复杂的多智能体协作 workflow，每一次 LLM 调用都需要携带越来越大的上下文窗口。



Agent Working Memory 的构成

系统提示词 (System Prompt)	定义角色、行为规则与约束条件	永久固定
工具定义 (Tool Definitions)	函数签名、参数描述与调用规范	半固定
多轮对话历史 (Conversation History)	用户输入、模型输出与工具返回值	持续增长
Agent 间状态传递 (Inter-Agent State)	上下游 Agent 的输出与中间结果	动态变化

典型场景: 10 个 Agent x 20 轮交互 = 百万级 token 的工作记忆

核心矛盾：重复 Prefill 的代价

当前问题

每次 LLM 调用独立处理完整上下文

结果

大量重复 Prefill → 显存碎片化 + 高延迟

相同上下文被反复计算的示意：

调用 1	重复上下文	新
调用 2	重复上下文	新
调用 3	重复上下文	新
调用 N	重复上下文	新

无 KV Cache 持久化

TTFT 慢 136x

首 token 延迟急剧恶化

MCP 动态加载工具

命中率 85% → 0%

工具列表变化导致缓存失效

JSON 序列化不确定性

TTFT 慢 65%

字段顺序/空格差异破坏前缀匹配

02

KVCache 即物理工作记忆

从内存管理到一等系统资源

KVCache = Agent 的物理工作记忆

KVCache 不只是推理引擎的内部数据结构，
它是智能体系统最核心的"物理工作记忆"载体

30 年前的数据库

数据库的 Buffer Pool

DB 从通用 OS 中独立 → 自己管理内存

现在的 AGENTIC 推理

Agentic 系统的 KVCache

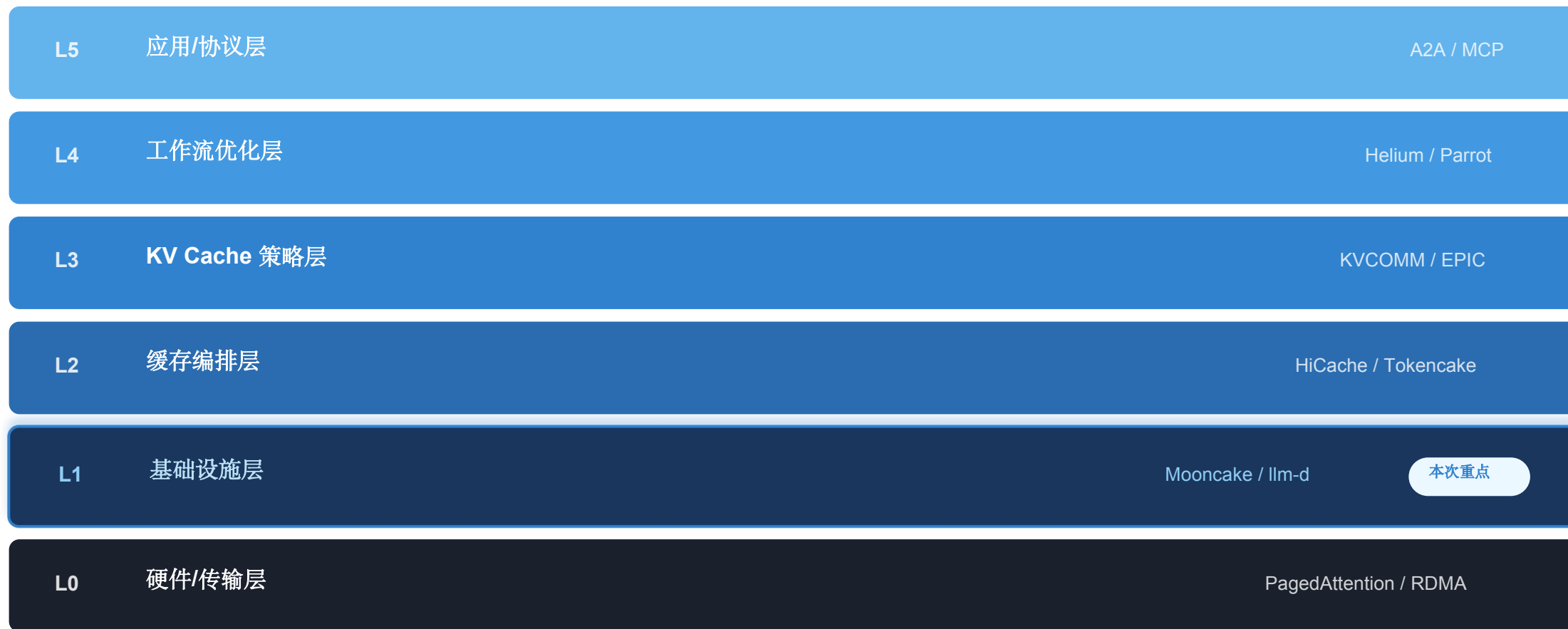
Agentic 推理从通用 LLM Serving 中独立 → 自己管理 KVCache

数据库的 Buffer Pool ↔ Agentic 系统的 KVCache — 历史的类比，未来的方向

Agentic vs 传统推理工作负载

交互模式	单轮/少量多轮	数十轮 (LLM → Tool 循环)
KV Cache 生命周期	请求内释放	需跨工具调用持久化
Prompt 结构	相对固定	高度模块化 + 动态组合
前缀共享	同一用户多轮	多 Agent 间共享
输入/输出比	~10:1	>100:1
workflow 拓扑	线性	DAG / 动态图

KV Cache 优化的分层技术栈



↑ 越往上越贴近应用语义 | 越往下越贴近硬件 ↓

03

Mooncake: KVCache-Centric 分离架构

一次计算，全局共享

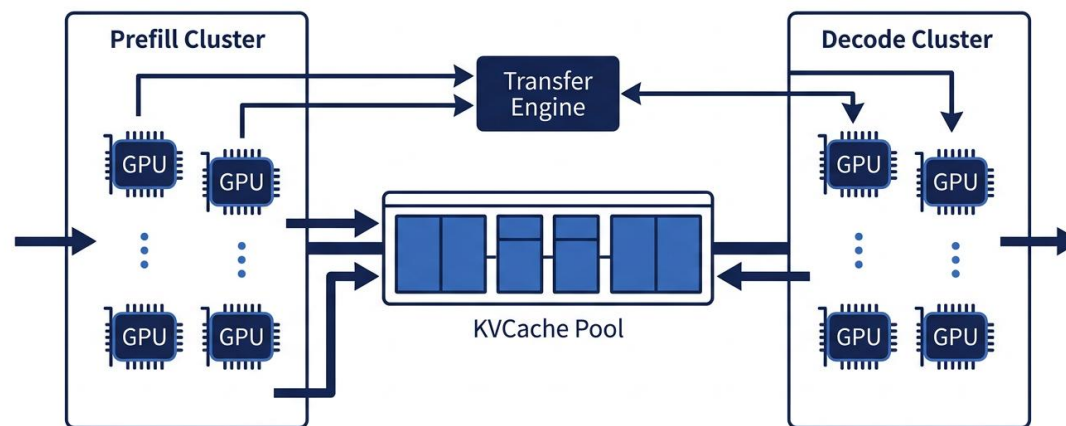
Mooncake 整体架构

KVCache-Centric 分离架构 [FAST'25](#)

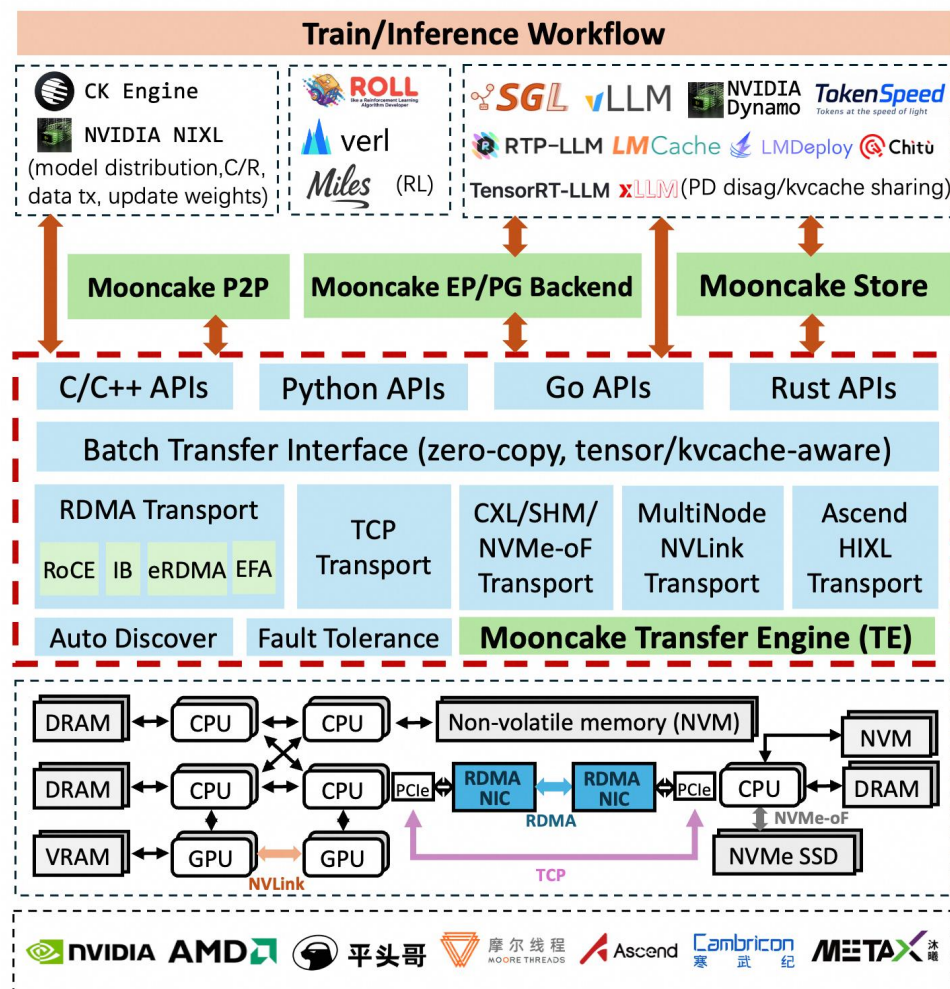
全局共享 KVCache Pool

Prefill 与 Decode 解耦

525% 吞吐提升 (vs 基线)



Mooncake 完整架构全景



★ Starred 5.5k

Contributors 233

Contributors 233

| Eco-system

Mooncake 开源社区与生态

核心团队

核心贡献者

阿里巴巴 · 清华大学

Approaching AI · Moonshot AI

协作团队

广泛的开源社区

来自全球的开发者持续贡献

github.com/kvcache-ai/Mooncake

生产就绪



K8S 部署

云原生容器编排



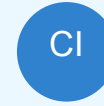
whl 包

Python 标准安装



Nightly Build

每日自动构建



端到端 CI

全流程自动化测试

Mooncake 社区影响力与生态支持

Feature	Project	Type	Transfer	EP/Torch Backend	Store	Ckpt Engine
Inference	SGLang (Omni)	Inference	✓	✓	✓	✓
	vLLM V0	Inference	✓	✗	✓	✓
	vLLM V1 (Omni)	Inference	✓	WIP	✓ (native/LMCache/Nixl)	✗
	LMDeploy	Inference	✓	✗	✓	✗
	TensorRT-LLM	Inference	✓	✗	✓	✗
	Chitu	Inference	✓	✗	✗	✗
	xLLM	Inference	✓	✗	✓	✗
	RTP (Alibaba)	Inference	✓	✗	✓	✗
Middleware	KVCM (Alibaba)	Middleware	✓	-	✓	-
	TBase (Ant)	Middleware	✓	-	✗	-
	Dynamo	Framework	✓ (w/ Nixl)	✗	✗	✗
	LMCache	Middleware	✗	-	✓	-
	TransferQueue	Middleware	-	-	✓	-
RL Post-Training	Slime/Miles	RL	WIP	-	✓	-
	ROLL (Alibaba)	RL	✗	-	✓	-
	Verl	RL	✗	-	✓ (w TransferQueue)	-

12+

集成项目

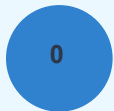
4

模块维度 (Transfer/EP/Store/Ckpt)

3

场景覆盖 (推理/中间件/RL训练)

Transfer Engine: 全链路零拷贝



全链路零拷贝

端到端零拷贝数据传输，消除冗余内存搬运



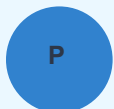
多网卡池化

聚合带宽 8×400Gbps，拓扑感知智能路由



容错与负载均衡

自动故障切换，动态负载分配



多协议支持

RDMA / TCP / SHM / NVLink 统一抽象



动态拓扑

比 NCCL 更灵活，支持动态拓扑变更



高性能传输

面向 KVCache 传输场景深度优化

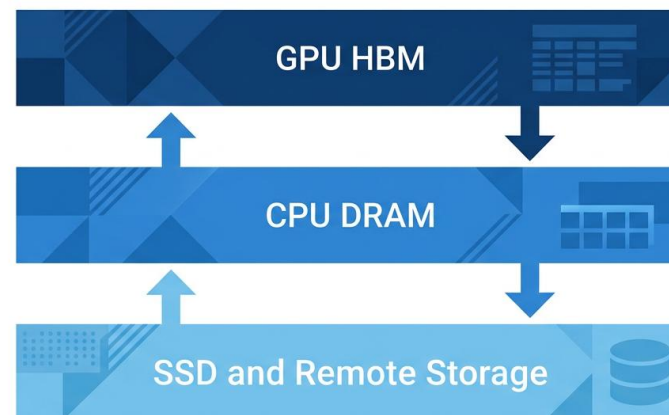
KVCache Store: 多级缓存存储

透明多级缓存: **VRAM** → **DRAM** → **SSD** → **Remote**

Key 由 Value Hash 计算, 无需 Update

支持 Lease 机制, 全局资源管理

零拷贝接口, 批量 API, 切片传输



支持 3FS 高性能存储后端, USBIO 原生接口

Store 层次化存储: 3FS 集成详解

1 挑战: 并发读性能不足

3FS 单次读取吞吐有限, 无法满足高并发 KV Cache 读取需求



1 方案: BatchGet 多线程并发

批量请求合并 + 多线程并发获取, 大幅提升读取吞吐

2 挑战: POSIX/FUSE 开销大

上下文切换 + 数据拷贝导致延迟高, CPU 开销大



2 方案: 3FS 原生 USBIO API

绕过 FUSE 直接调用原生 API + 并行 I/O 优化, 降低延迟

3 挑战: 元数据一致性

客户端管理元数据时, 多节点一致性难以保障



3 方案: Master 节点集中管理

元数据迁移至 Master 节点, 支持双类型: 磁盘型 + 内存型

Prefill-Decode 分离架构

今天是分离架构的时代! (Today is a Disaggregation Era!)

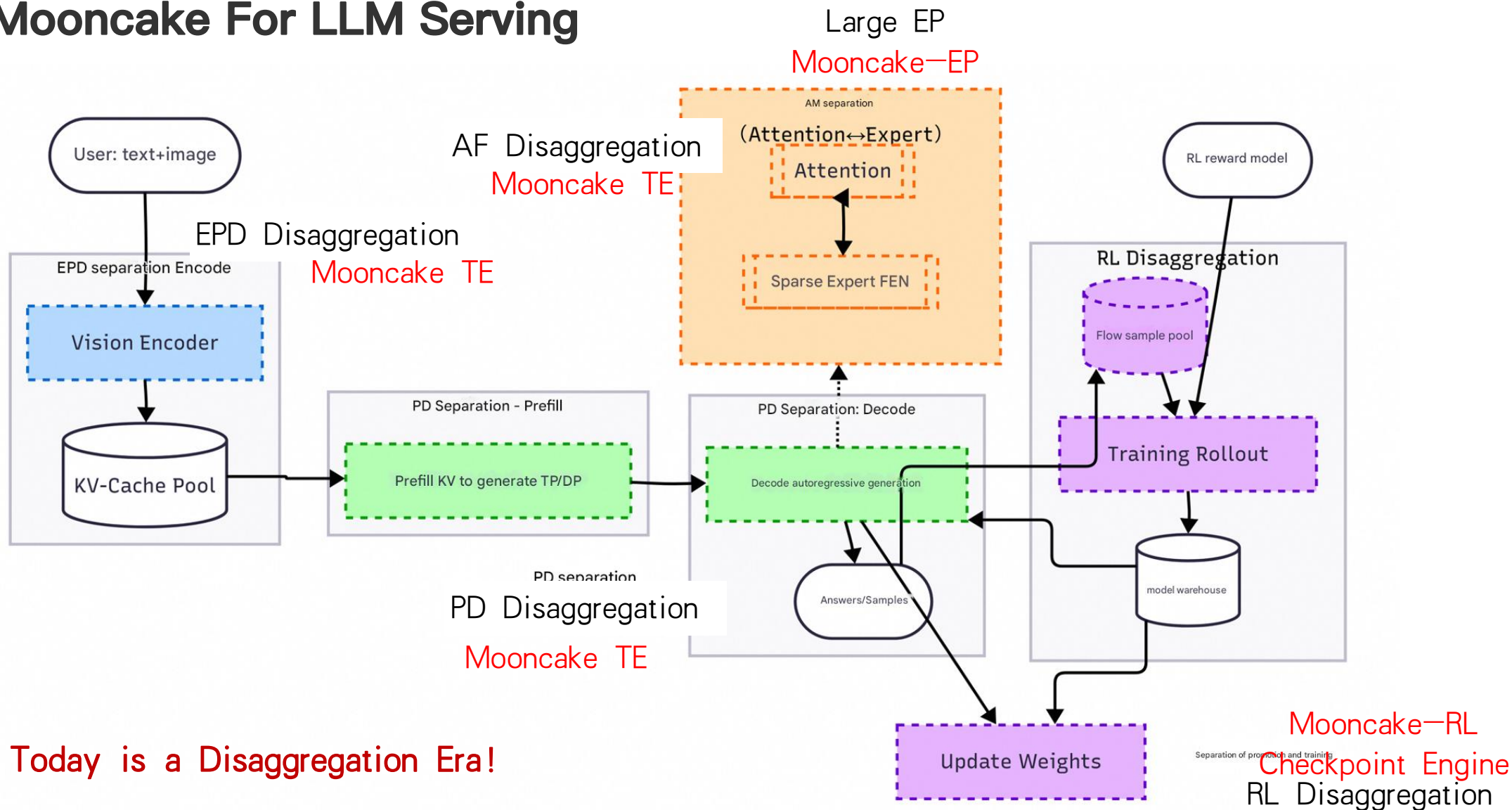
Pattern 1: Transfer Engine Based PD



Pattern 2: Store Based PD



Mooncake For LLM Serving



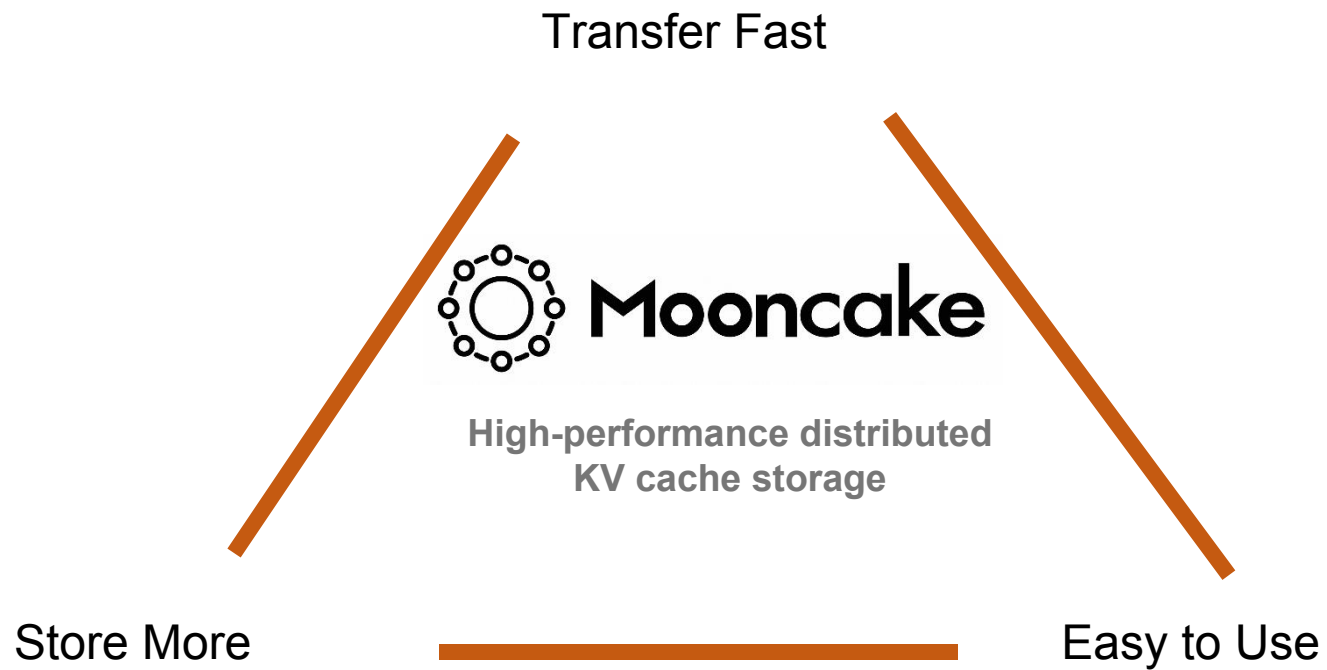
Today is a Disaggregation Era!

04

社区生态与集成

开源协作 · 硬件厂商合作 · Highlights

Mooncake Architecture

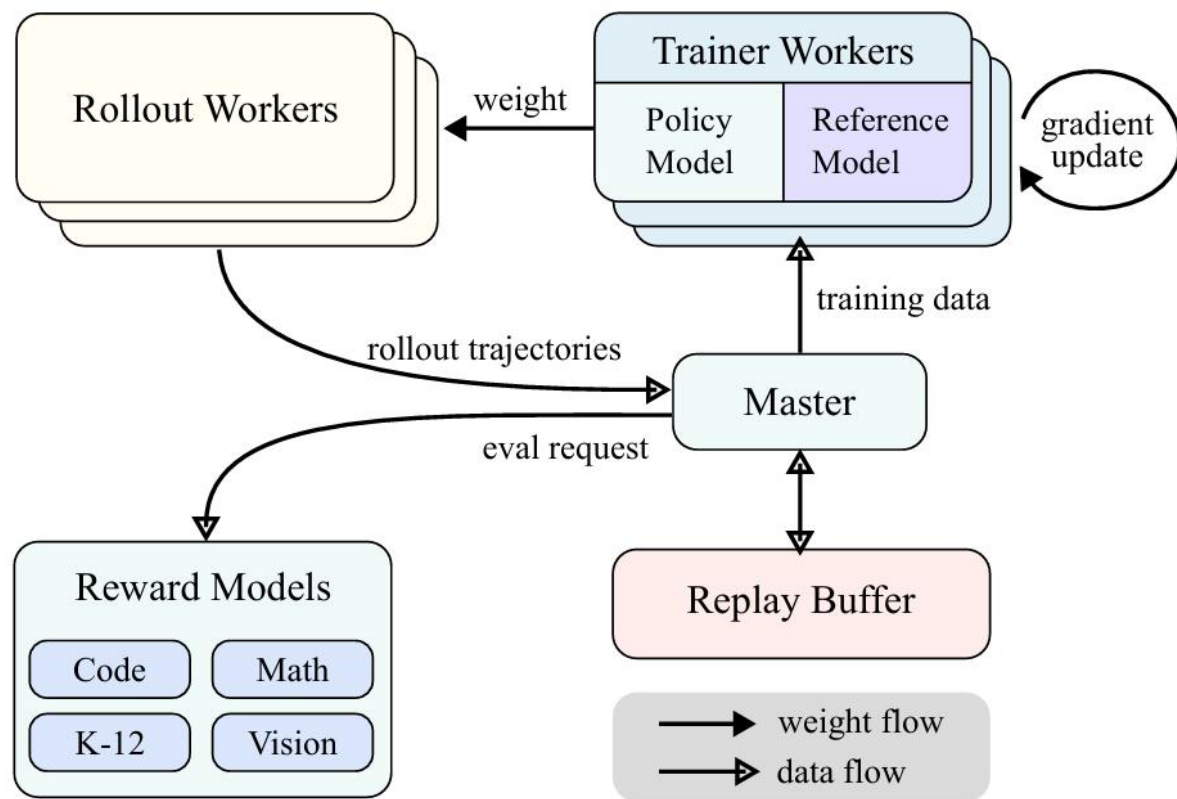


- Elastic, Shared, and Multi-layer KV Cache
- Memory Allocator Optimized for LLM Inference

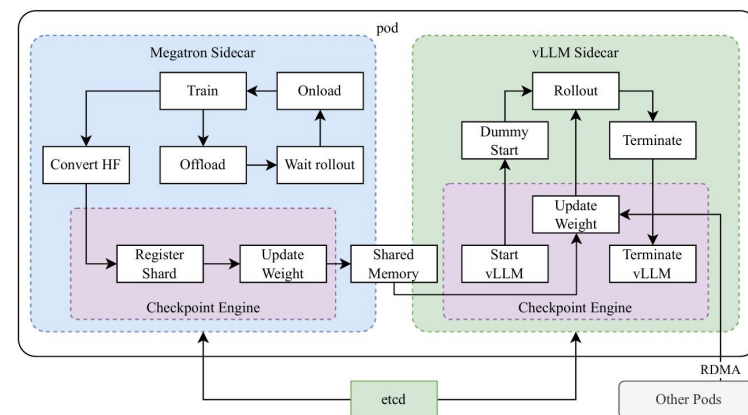
- Extensive and user-friendly APIs

Recent Highlight 1: Weight Update in RL

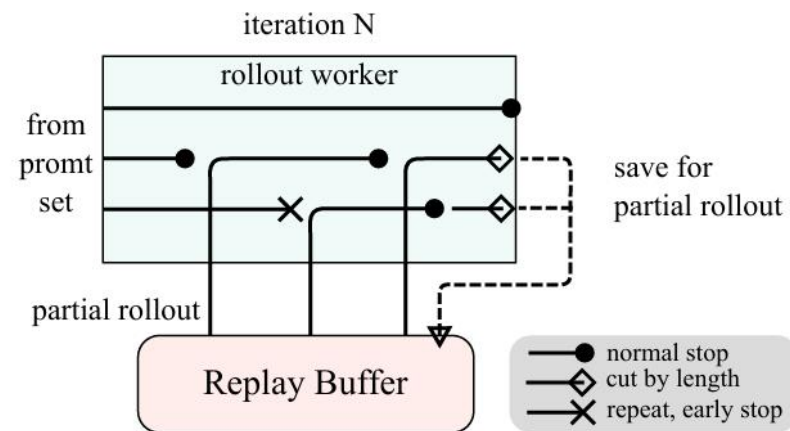
<https://github.com/MoonshotAI/checkpoint-engine/>



(a) System overview



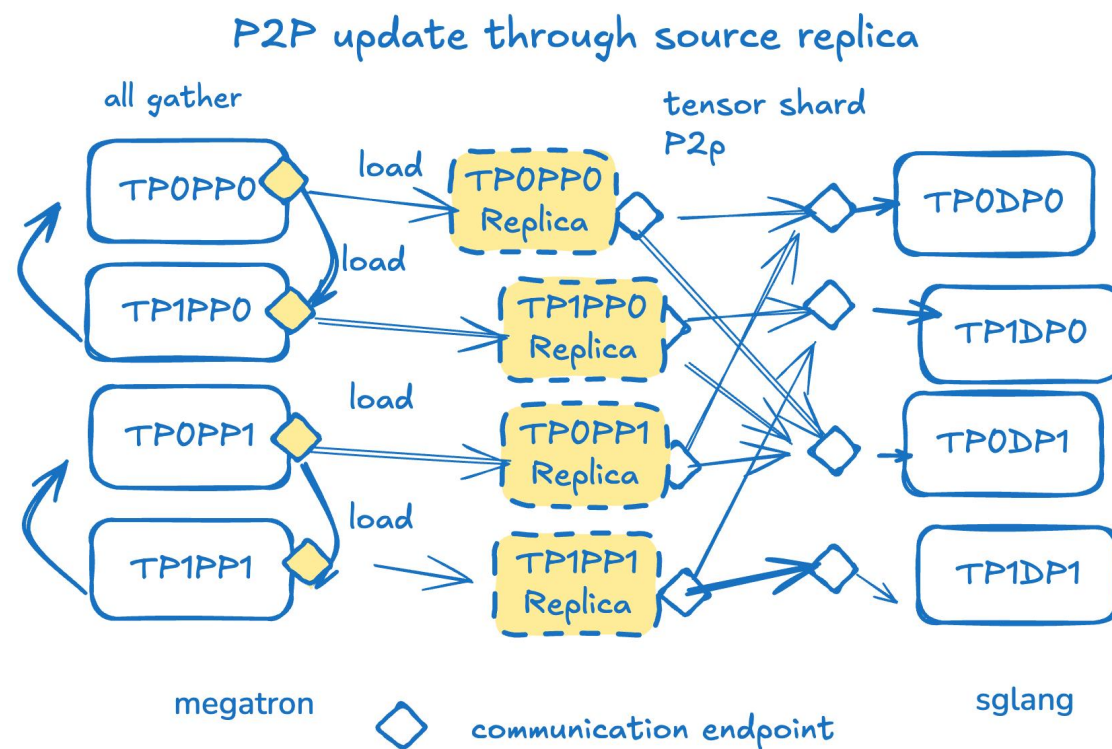
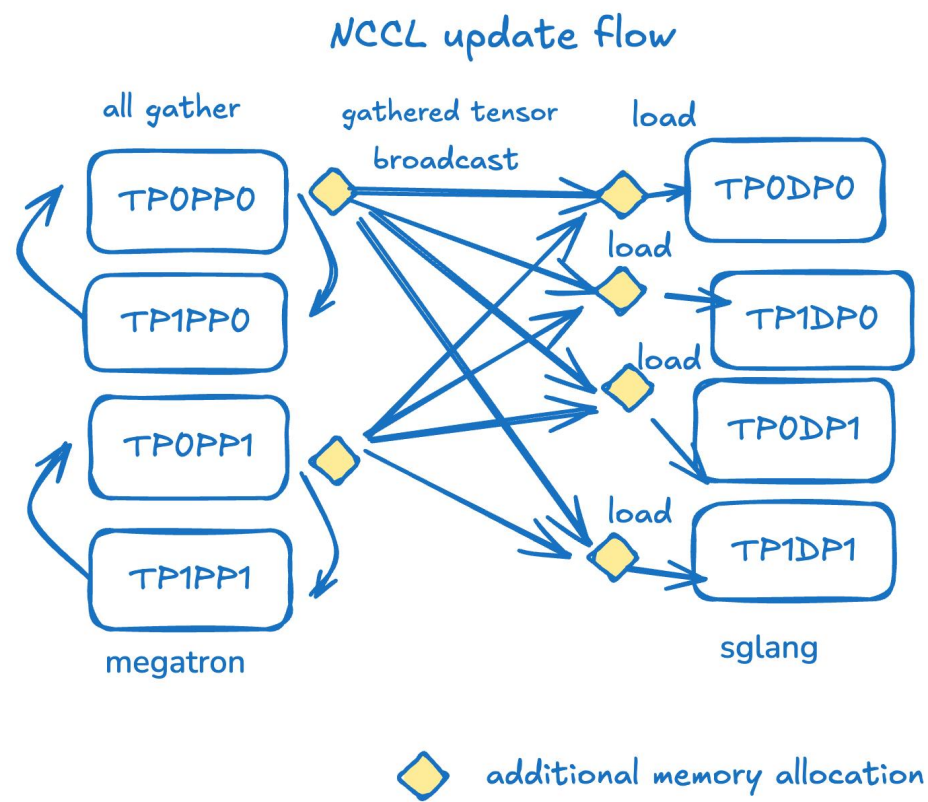
Fast Checkpoint Transfer



(b) Partial Rollout

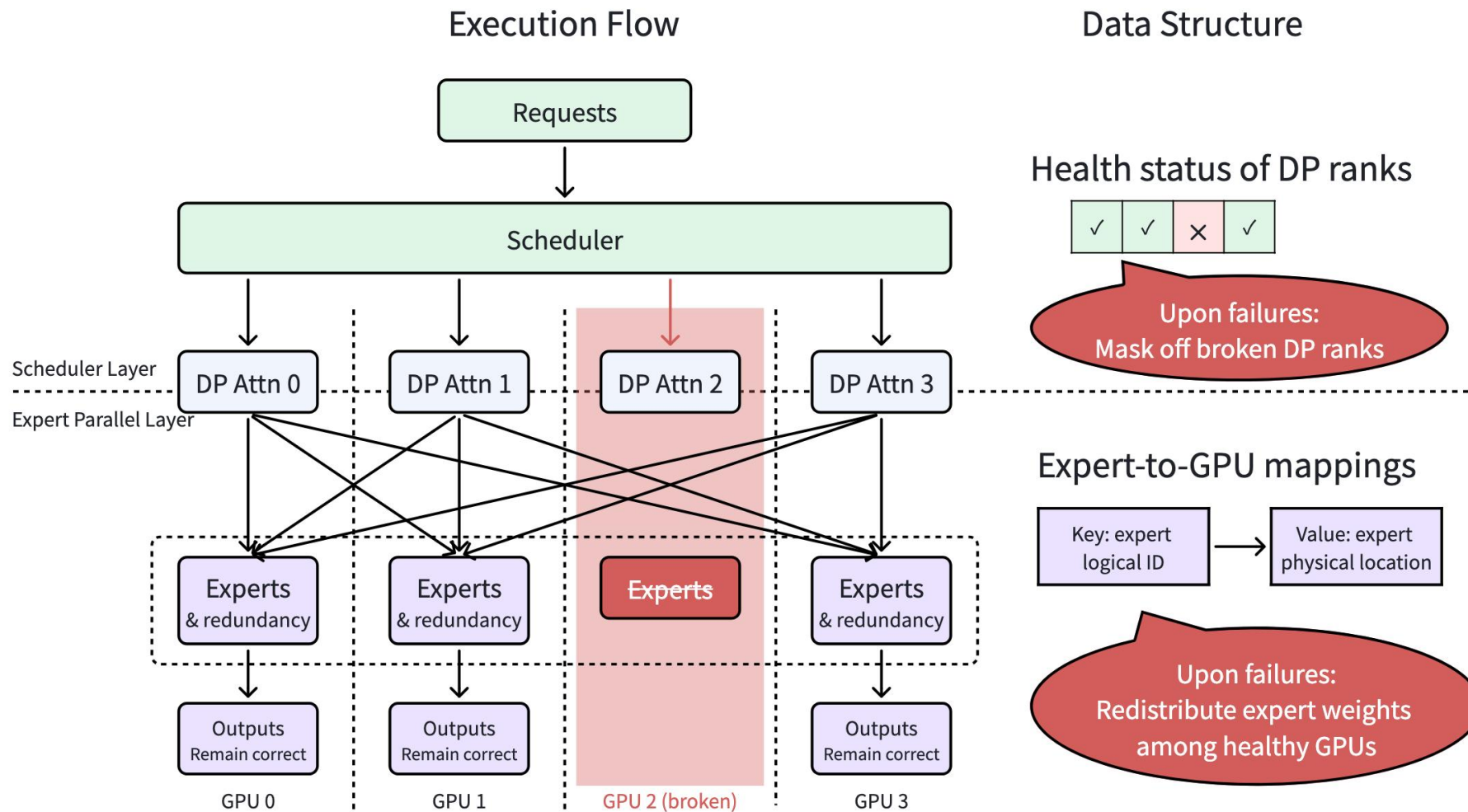
Recent Highlight 1: Weight Update in RL

[Updating 1T parameters in seconds — P2P weight transfer in Large Scale Distributed RL — LMSYS Blog | LMSYS Org](#)

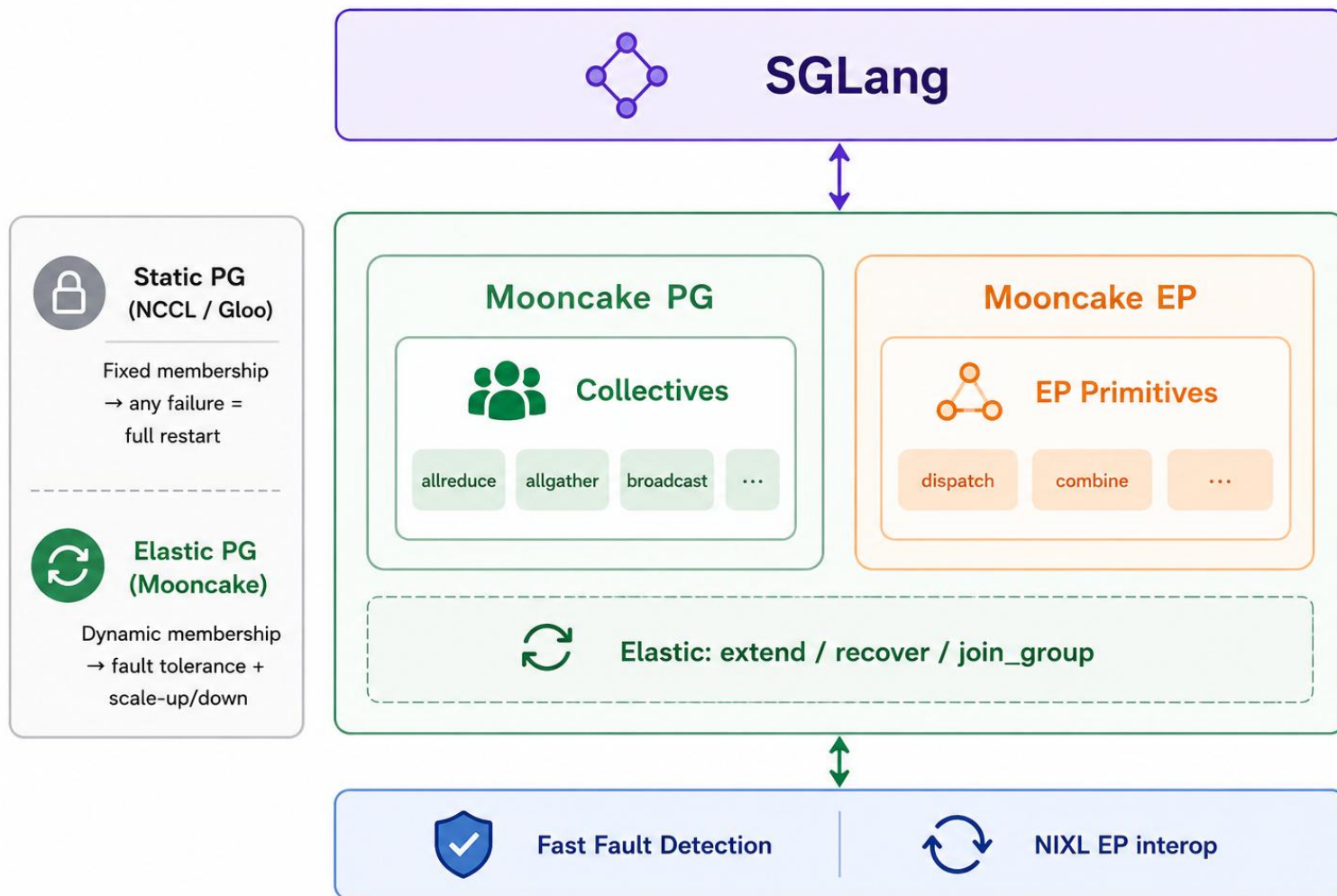


Recent Highlight 2: Elastic EP

[Elastic EP in SGLang: Achieving Partial Failure Tolerance for DeepSeek MoE Deployments – LMSYS Blog](#) | [LMSYS Org](#)

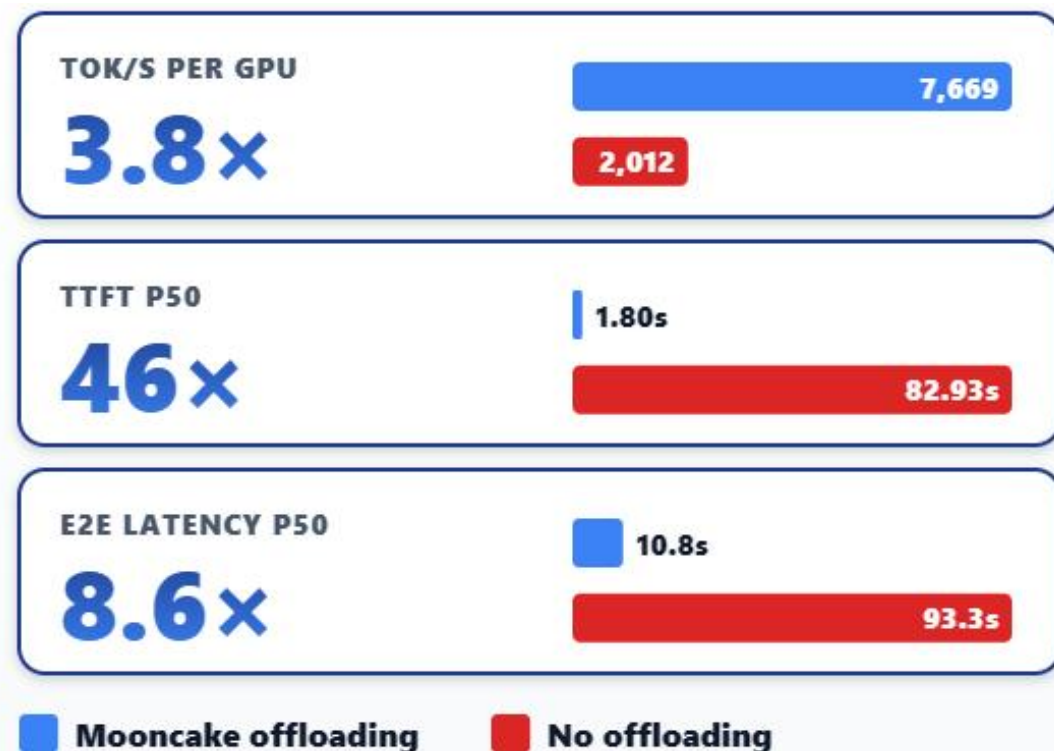
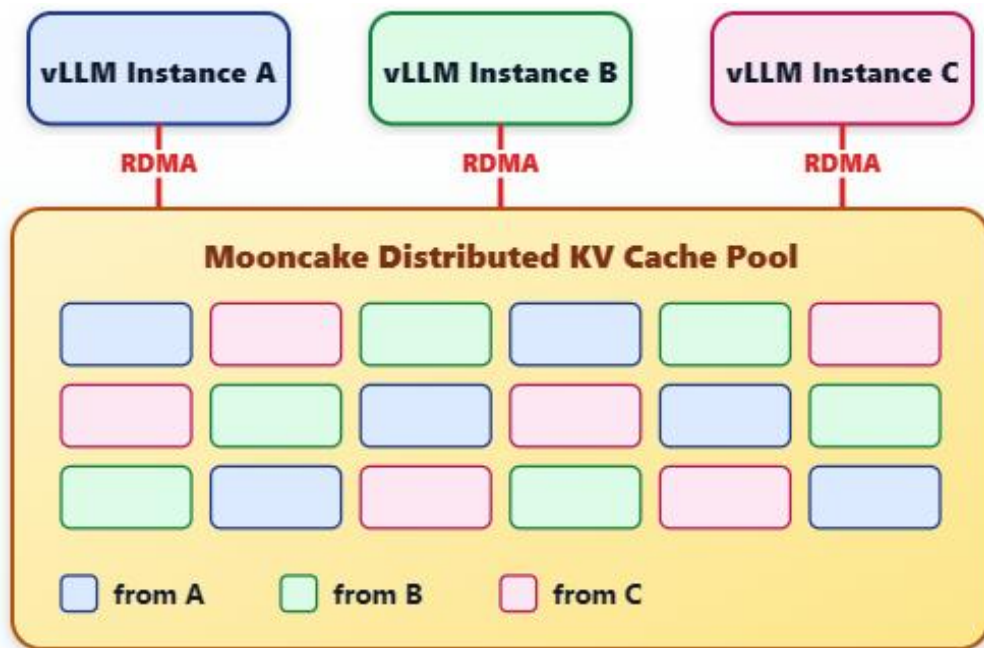


Recent Highlight 2: Elastic EP



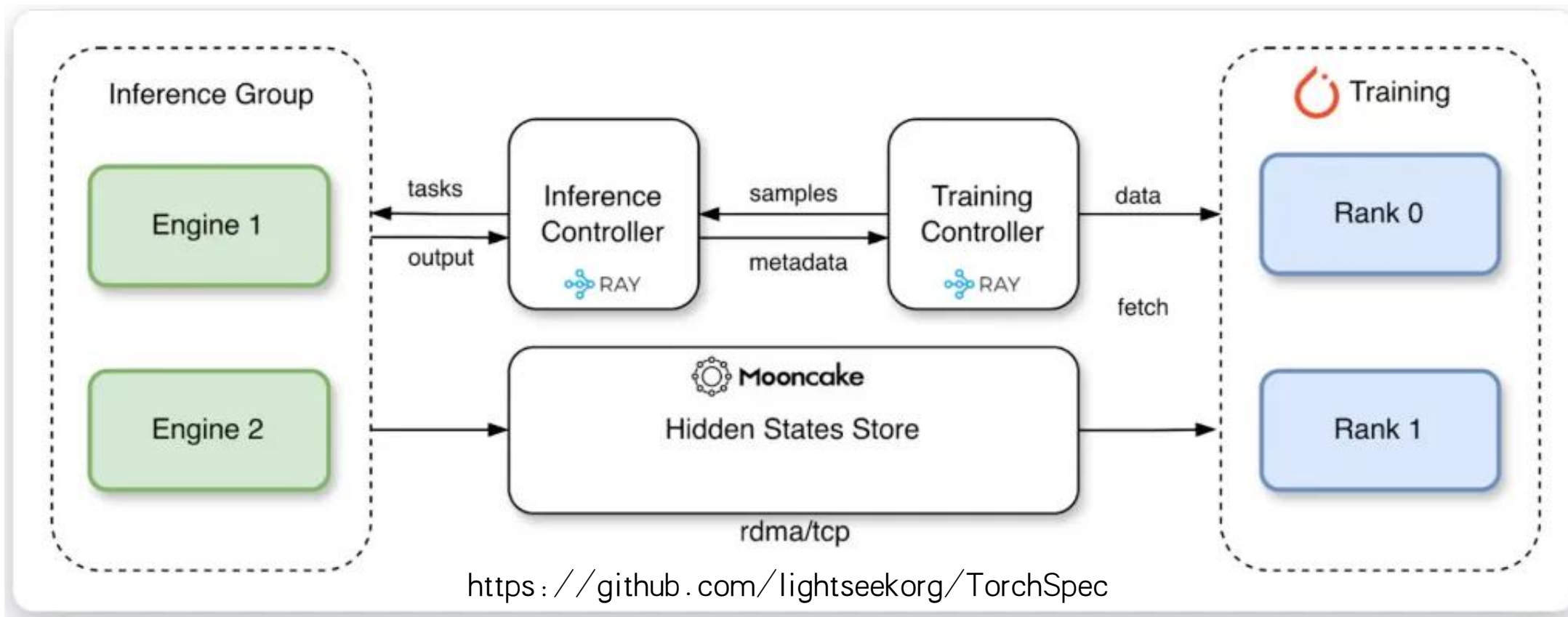
Recent Highlight 3: Direct Integration with vLLM

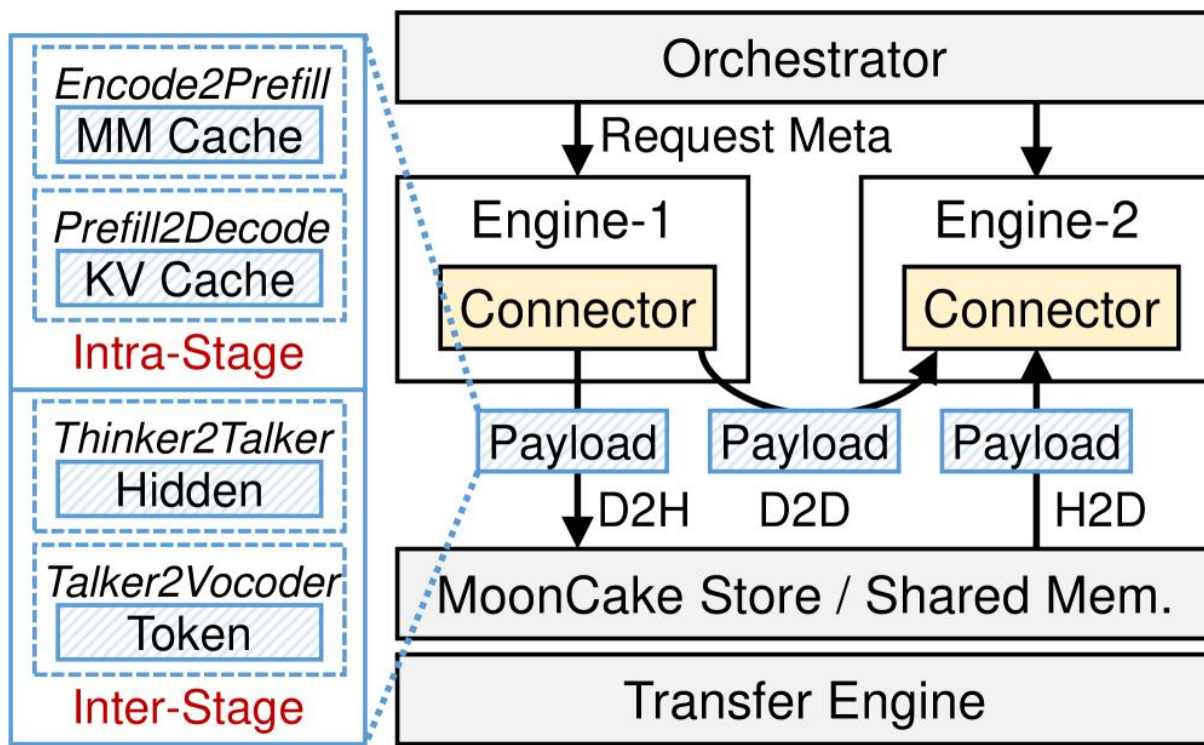
[Serving Agentic Workloads at Scale with vLLM x Mooncake | vLLM Blog](#)



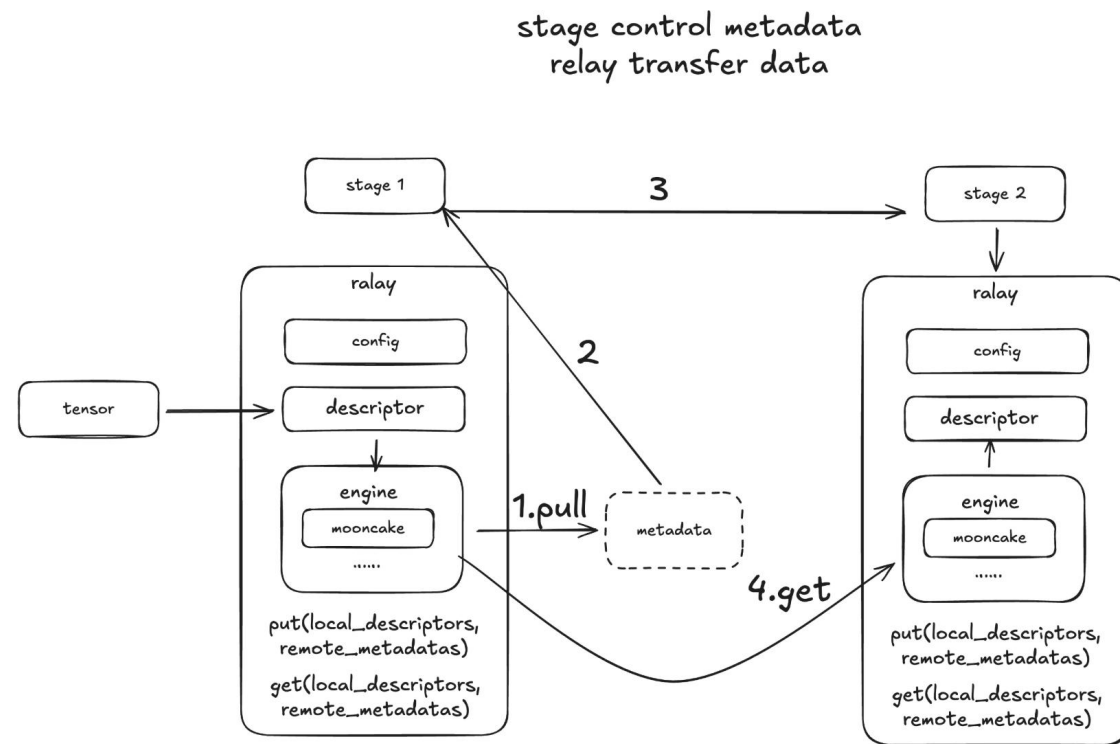
Recent Highlight 5: Draft Model Training

- TorchSpec is a PyTorch-based decoupled speculative decoding training framework.
- Mooncake is responsible for directly, efficiently, and continuously streaming hidden states from the inference side to the training side, allowing training and inference to be truly decoupled.





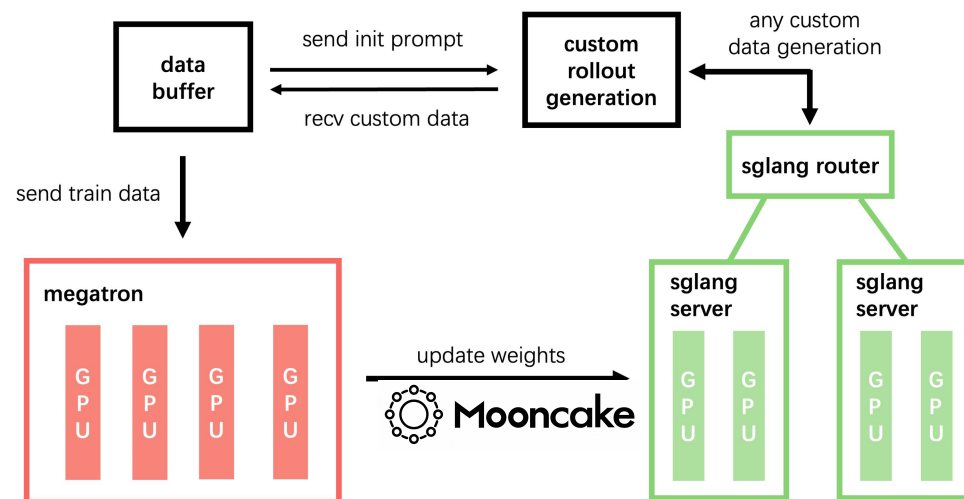
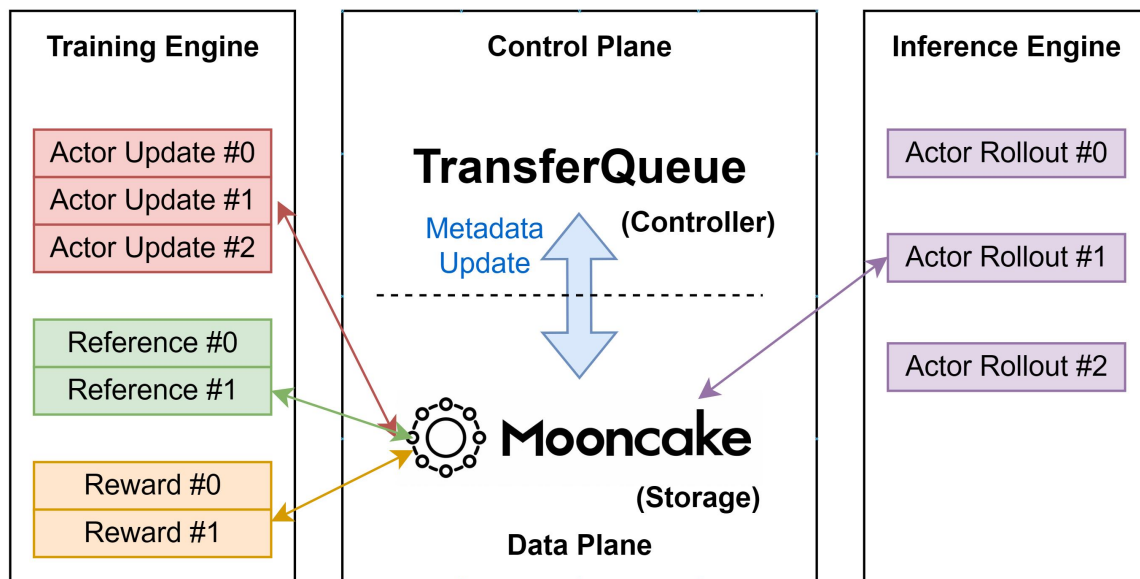
SGLang-Omni



Recent Highlight 6: RL

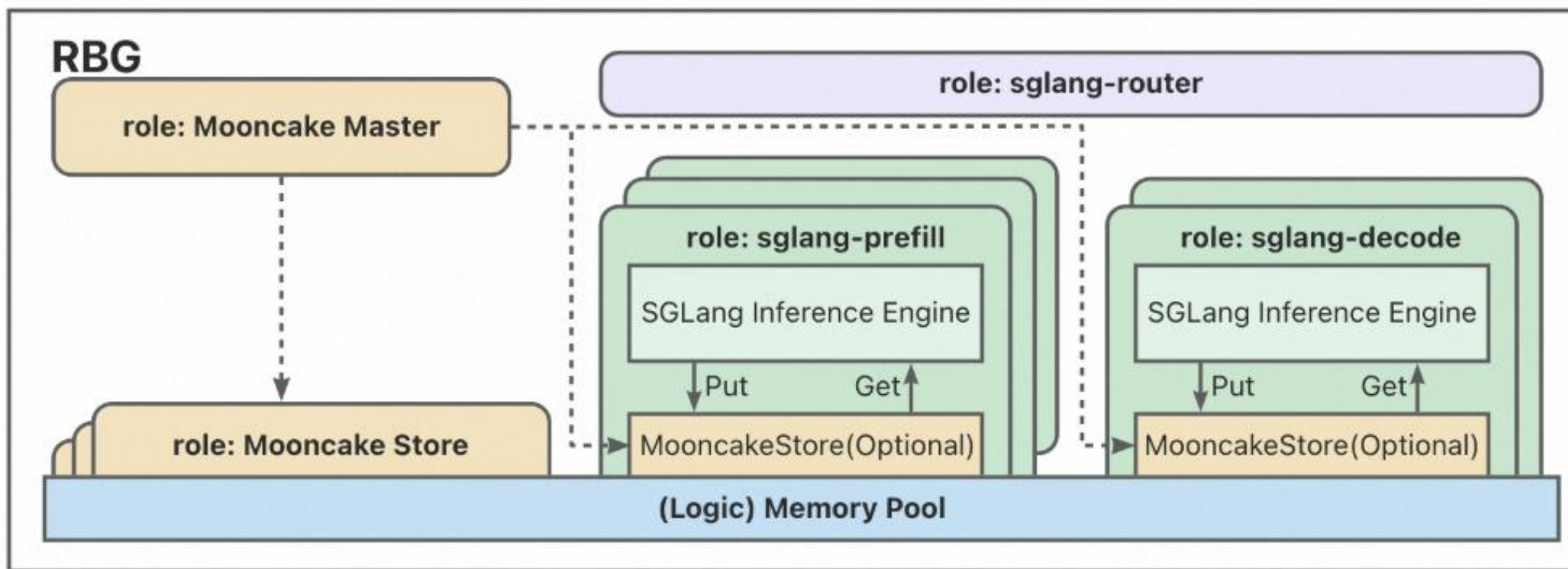
Integration with TransferQueue (Ver1), Slime (WIP), Roll

- Serves as a high-performance distributed storage backend in disaggregated RL architectures for model weight synchronization.
- Helps post-training frameworks decouple complex task dependencies, enabling large-scale, asynchronous, and non-blocking data scheduling across heterogeneous devices.



Recent Highlight 7: Large Scale Deployment

RBG+SGLang+Mooncake



Mooncake X NVIDIA

Transfer Engine + Nixl:

- ❑ Transfer Backend
- ❑ Bazel Build

Dynamo + Mooncake Store:

- ❑ Rust API
- ❑ KVBM Integration

Support Cuda13/MNNVL/GB200/GB300:

Mooncake Trace in Dynamo Bench:

Dynamo KV Router + SGLang/vLLM + Mooncake Store:

- ❑ KV Events API Standardization (#1527)
- ❑ KV Event Integration (#1408)

Add Mooncake backend in NIXL #169

 **Merged** [tstamler](#) merged 19 commits into [ai-dynamo:main](#) from [alogfans:mooncake-support](#) on May 9, 2025

 Conversation 26  Commits 19  Checks 5  Files changed 13



alogfans commented on Apr 19, 2025 · edited

Contributor · ...

Mooncake transfer engine is a high-performance, zero-copy data transfer library. To achieve better performance in NIXL, we have designed a new backend based on Mooncake transfer engine.

[RFC]: Dynamo KVBM + Mooncake Store Integration Design #1979

 Open

 #1993



Dayuxiaoshui opened 2 weeks ago · edited by Dayuxiaoshui

Edits · Contributor · ...

  **Fix NVLink IPC offset corruption for sub-allocated GPU tensors** ✓  high priority  run-ci  Transfer Engine 15

#1622 by [ishandhanani](#) Contributor was merged on Mar 12 · Approved

  **fix: check MC_INTRANODE_NVLINK before HCA auto-detection fallback** ✓  run-ci  Transfer Engine 4

#1612 by [ishandhanani](#) Contributor was merged on Mar 5 · Approved 1 task done

[dynamo](#) / [lib](#) / [bench](#) / [agent_trace_to_mooncake.rs](#)

[dynamo](#) / [lib](#) / [bench](#) / [src](#) / [coding](#) / [mooncake.rs](#)

[dynamo](#) / [lib](#) / [bench](#) / [kv_router](#) / [mooncake_shared.rs](#)

9. Open Discussion Points

1.  **Batch API gap** (RESOLVED): PR  [\[Store\]\[Rust\] Add batch_put_from, batch_get_into, batch_is_exist wrappers #1993](#) merged. Rust wrapper now exposes `batch_put_from` / `batch_get_into` / `batch_is_exist`. Migrated in `client.rs` on 2026-05-04.
2. **Key format unification**: KVBM `SequenceHash` (base58) vs SGLang `HiCache` key format in Mooncake. Should we unify for KV Router cross-system block queries?
3. **Metadata server selection**: HTTP Python server for local test; production should reuse dynamo existing etcd?
4.  **Phase 2 RDMA trigger timing** (COMPLETE): `register_layout` / `unregister_layout` + `with_zero_copy` builder implemented. Zero-copy batch path uses `batch_put_from` / `batch_get_into` for contiguous layouts.

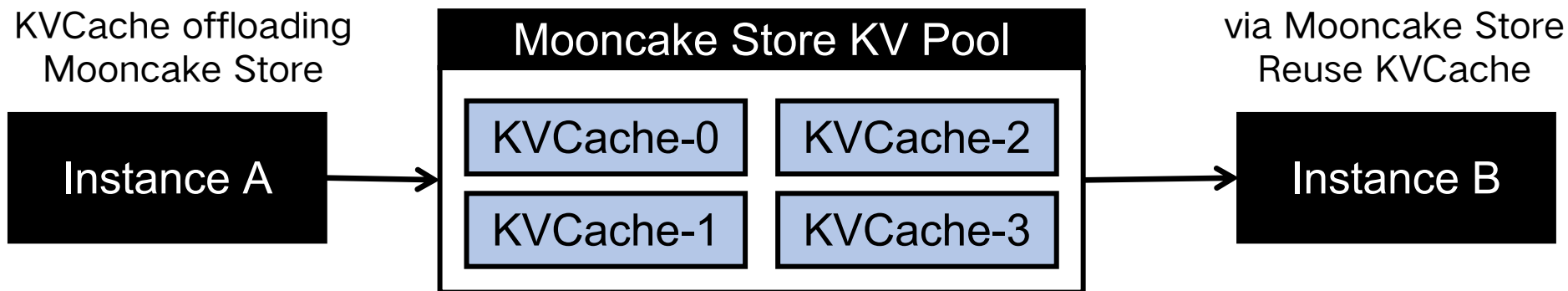
Mooncake X vLLM

MooncakeStoreConnector:

vLLM intra-node **KVCache sharing**

Mooncake Distributed KVCache Storage

KVCache offloading
Mooncake Store



- May 7, 2026: 🚀 vLLM officially features [Mooncake Store](#) — a deep dive into how Mooncake's distributed KVCache engine supercharges vLLM inference with high-throughput, memory-efficient, cross-instance KV cache sharing!

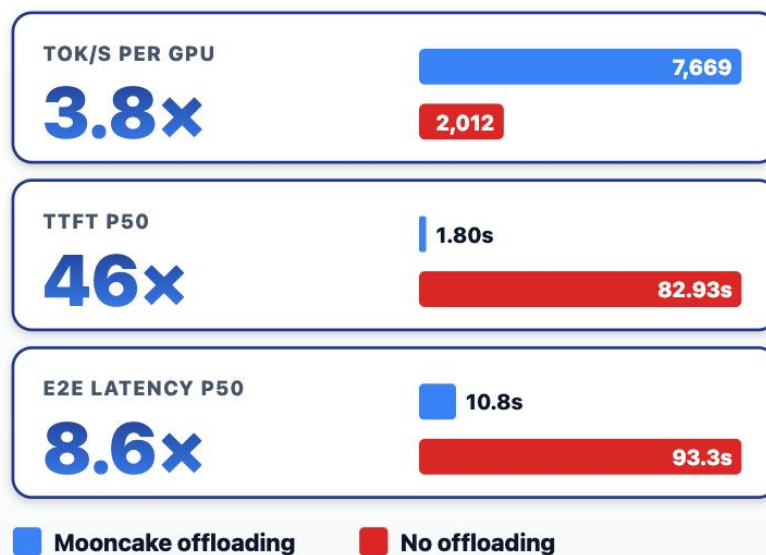
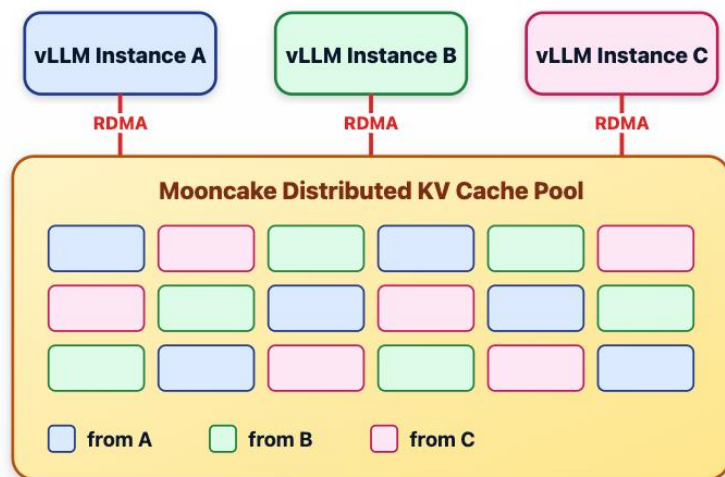
TP、PP、CP:

- Check if all shards of a block's KVCache have been hit.
- Prefix Cache: Find the first missed block and reuse the blocks preceding it.

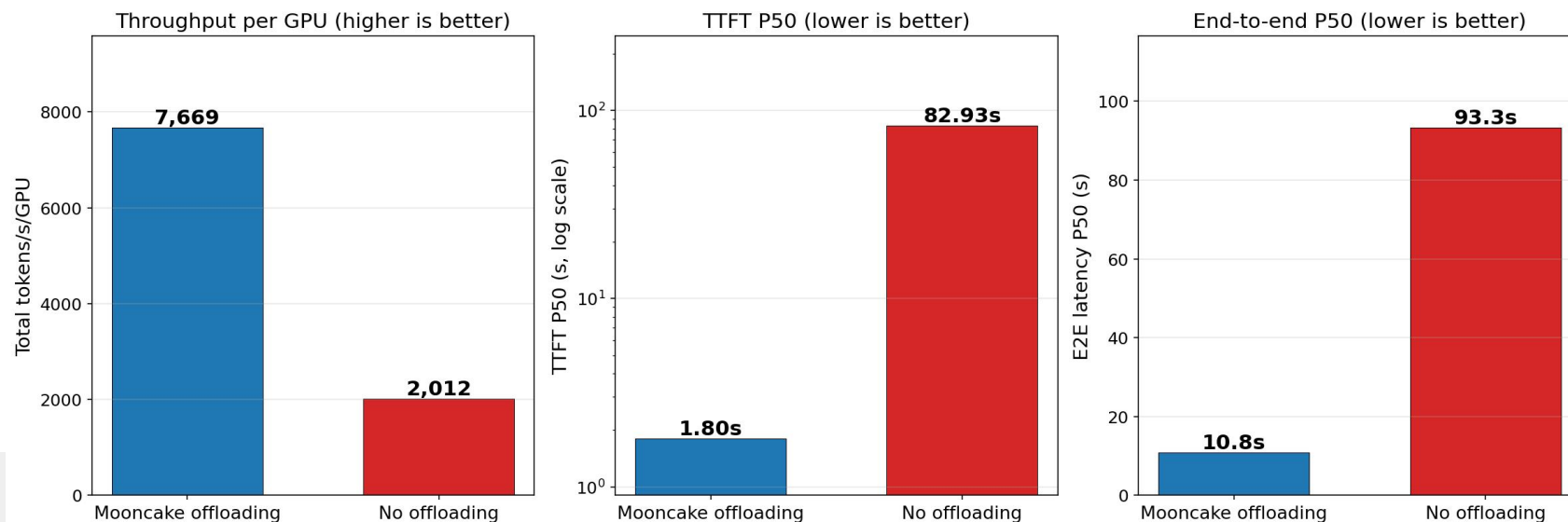
```
# Expand keys for all TP ranks
multi_tp_keys = keys[:]
for i in range(1, min(self.tp_size, self.num_kv_head)):
    for item in keys:
        new_str = item.replace("@tp_rank:0", f"@tp_rank:{i}", 1)
        multi_tp_keys.append(new_str)
```

```
# Expand keys for all PP ranks
pp_base_keys = multi_tp_keys.copy()
for i in range(1, self.pp_size):
    for item in pp_base_keys:
        new_str = item.replace("@pp_rank:0", f"@pp_rank:{i}", 1)
        multi_tp_keys.append(new_str)
```

|

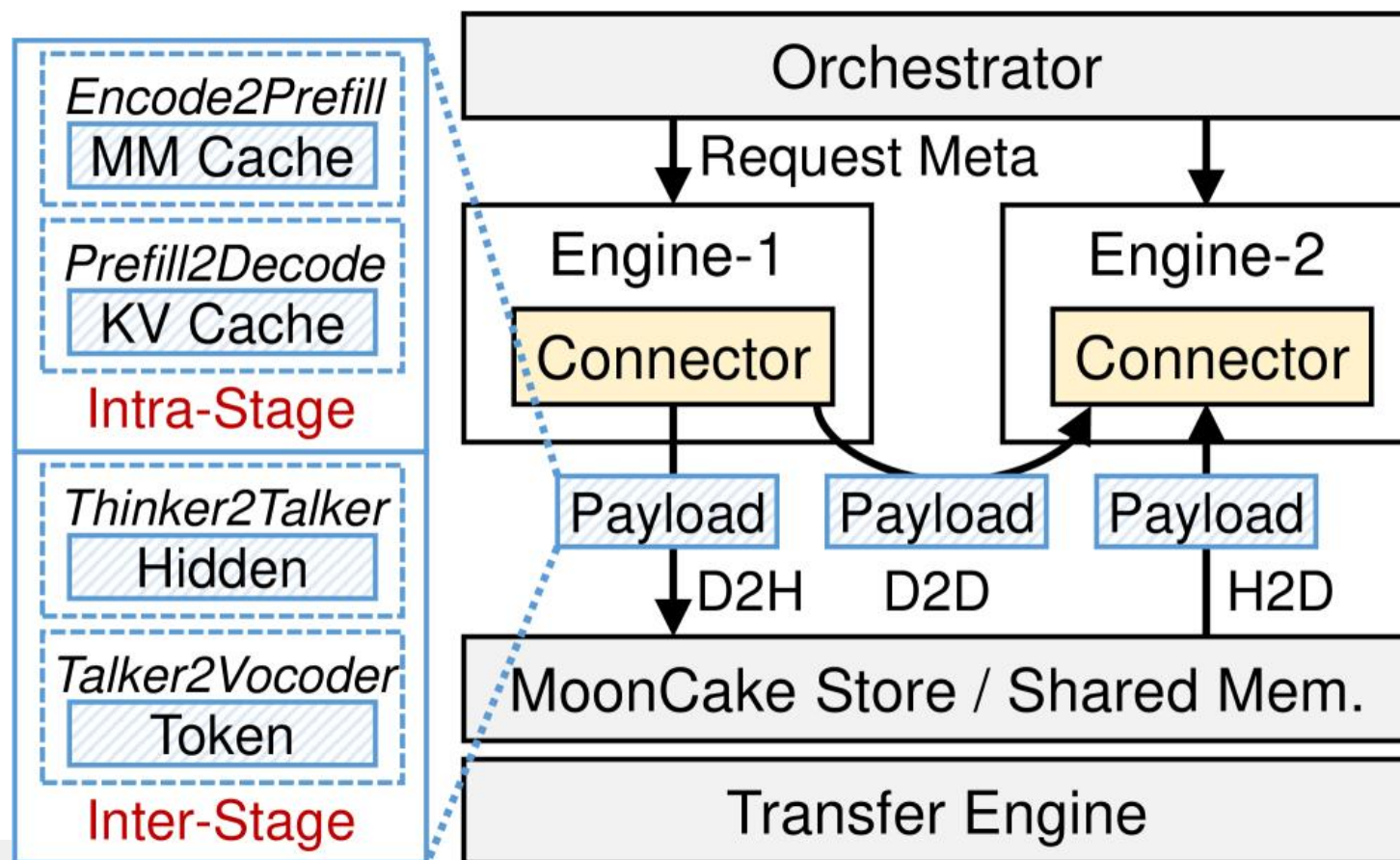


Mooncake offloading vs no offloading — Kimi-K2.5-NVFP4, codex_swebenchpro, 1P1D (12 GPUs, 50 convs)
Mooncake offloading delivers 3.8x tok/s/GPU, 46x faster TTFT, 8.7x faster E2E



Mooncake X vLLM Omni

- Based on Mooncake store / transfer engine , vLLM-omni connector



Mooncake X vLLM Roadmap



dtccccc opened on Mar 13 · edited by dtccccc

Edits Contributor ...

The feature, motivation and pitch

Following the successful initial integration of Mooncake Transfer Engine into vLLM ([#24718](#) [#31034](#)), this RFC proposes a feature roadmap to unlock more advanced capabilities, enhance performance, and improve flexibility. The foundational work allows for basic KV cache offloading, but to fully leverage the potential of Mooncake and support more complex, large-scale inference scenarios, we need to implement more sophisticated parallelism strategies and optimizations.

Core Features:

- ☐ Mooncake store integration ( [\[RFC\]: Add Mooncake Store Connector for Shared KV Cache Reuse #38474](#)  [\[KV Transfer\] Add MooncakeStoreConnector for KV cache offloading via Mooncake distributed store #40900](#))
- ☒ Heterogeneous TP support ( [\[KVTransfer\]\[Mooncake\] Add heterogeneous TP support for disaggregated P/D in MooncakeConnector #36869](#))
- ☐ PP support
- ☐ Layer-wise Transfer
- ☐ Cross-Layer KV Support ( [\[P/D\]\[Mooncake\] Add cross-layer KV cache support to MooncakeConnector #41093](#))
- ☐ PCP/DCP Support
- ☐ hybrid attention
- Other improvements and fixes
 - ☐ better error handling
 - ☐ physical block != config block
 - ☐ different hidden size per layer
 - ☐ HND-NHD transfer
 - ☒ kv events & stats ( [\[P/D\]\[Mooncake\] Add KVConnectorStats for transfer observability #40414](#))

- vllm [[#36948](#)]
- [RFC] PD Disaggregation with MooncakeConnector Roadmap
- vllm [[#38474](#)]
- [RFC] Add Mooncake Store Connector for Shared KV Cache Reuse
- vllm [[#32733](#)]
- [RFC] Bi-directional KV cache transfers between P and D nodes
- vllm [[#30112](#)]
- [RFC] Elastic DP Rank Recovery: Graceful Handling of GPU Hardware Failures

Many features!

EPD, P2P Transfer, Update Weights



Mooncake+MetaX Collaboration: PRs & MACA GPU Support

Transport: MACA C500 Intra-Node P2P (PR #2059)

- maca_transport: native MetaX MACA C500 backend
- Intra-node GPU↔GPU P2P transfer path
- Pluggable inside Transfer Engine plugin layer
- Validated on multi-card MACA C500 server
- 21 review tasks resolved before merge
- Merged May 10, 2026 by Dayuxiaoshui

Expert Parallelism: MACA Standalone (PR #2309)

- MACA standalone device EP support
- Mooncake EP runs natively on MetaX GPUs
- Enables Expert-Parallel inference on MACA
- Mooncake EP run-ci pipeline passing
- Approved with 5 review iterations
- Merged Jun 4, 2026 by Dayuxiaoshui

Build & Docs: USE_MACA Flag Alignment (PR #1814)

- Aligns USE_MACA with MUSA GPU paths
- Unified build/docs across vendor backends
- -DUSE_MACA=ON CMake option in core build
- Source build via MetaX MACA SDK
- MACA_ROOT / MACA_HOME env discovery
- Merged Apr 9, 2026 (21 review tasks)

Store: GPU VRAM Pointer Robustness (PR #1892)

- Fix segfault in disk-replica / offload paths
- Robust handling of GPU VRAM pointers
- Improves Store reliability on MACA backend
- Approved with 5 of 7 review tasks
- Merged Apr 20, 2026 by LujhCoconut
- Cross-vendor GPU memory hardening

Integration: MACA/MetaX GPU Support (PR #2019)

- TransferEngine MACA/MetaX GPU integration
- Fix RDMA dmabuf registration on MACA
- End-to-end inference path validated
- 9 review tasks resolved before merge
- Merged May 11, 2026 by JoeZhang-0x000
- Brings MetaX into RDMA fast-path

Hardware Coverage: MetaX Muxi Family

- MetaX (沐曦) MACA GPUs — Supported
- Flagship target: MetaX C500 inference GPU
- Listed in build matrix alongside CUDA / MUSA
- -DUSE_MACA=ON enables source build
- Headers / libs under MACA_ROOT (default /opt/maca)
- Documented in getting_started/build.md

Build Matrix & Community Status

- MetaX-Tech listed as community contributor
- Logo featured on README contributors row
- 5 merged PRs spanning Transfer / EP / Store
- Active engineers: Dayuxiaoshui, JoeZhang-0x000
- Cross-team review with Mooncake core team
- Continuous CI coverage on MACA paths

☐

[EP] Add MACA standalone device EP support

✓

Mooncake EP

run-ci

Transfer Engine

#2309 by Dayuxiaoshui

Contributor

 was merged 2 days ago • Approved

☐

[TransferEngine] Add maca_transport for Metax MACA C500 intra-node P2P

✓

run-ci

Transfer Engine

#2059 by Dayuxiaoshui

Contributor

 was merged last month • Approved

21 tasks

☐

[TransferEngine][Integration] feat: add MACA/MetaX GPU support and fix RDMA dmabuf registration

✓

Installation

run-ci

Transfer Engine

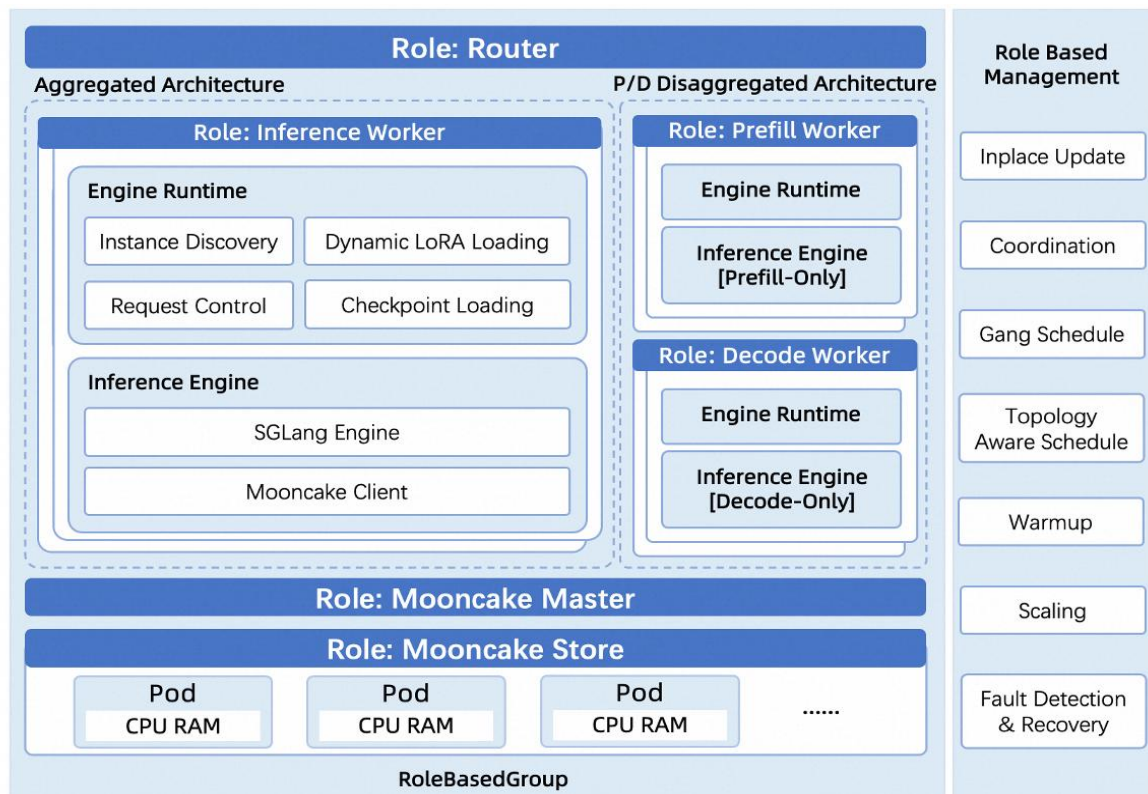
#2019 by JoeZhang-0x000

Contributor

 was merged last month • Approved

Source: github.com/kvcache-ai/Mooncake — PRs & Build Docs (2026)

生产部署: Router + RBG + SGLang + Mooncake Store



系统角色构成

- SGLang Router: 统一入口与流量调度, 智能分发至 Prefill 或 Decode 后端
- Prefill Worker: 计算密集型, 负责 Prompt 前向计算并生成初始 KVCache
- Decode Worker: 延迟敏感型, 依赖 KVCache 进行 Token 逐个解码
- Mooncake Store/Master: 独立外置存储角色, 持久化存储 KVCache

协同工作流

- 所有角色通过 RBG 进行紧密集成与协同
- 利用 RDMA 网络在计算节点与 Mooncake 存储节点间高速传输 KVCache
- Router 根据负载动态调度, 实现动态批处理与连续批处理

系统角色: SGLang Router 入口 → Prefill/Decode Worker → Mooncake Store/Master 独立存储

协同工作流: 所有 RBG 进行密集通信 → 高速 RDMA 网络 → 节点高速传输 KVCache

05

实战: OpenClaw + Mooncake

多会话推理尾延迟优化

OpenClaw + Mooncake: 多会话推理架构

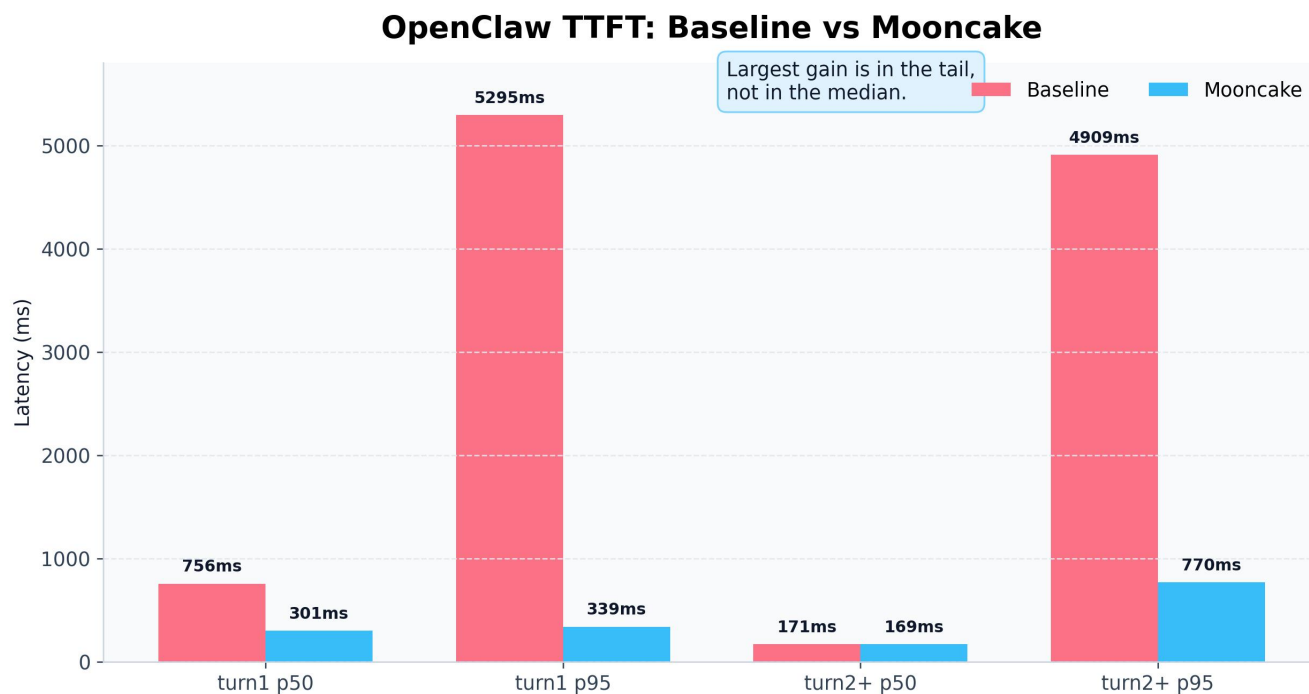
架构流程



测试配置

参数	值
模型	Qwen3-14B
会话数	2 独立会话
每会话轮次	4
调度方式	Round-robin
初始上下文	~24,000 字符
每轮新增	~8,000 字符

OpenClaw TTFT 性能对比



关键指标

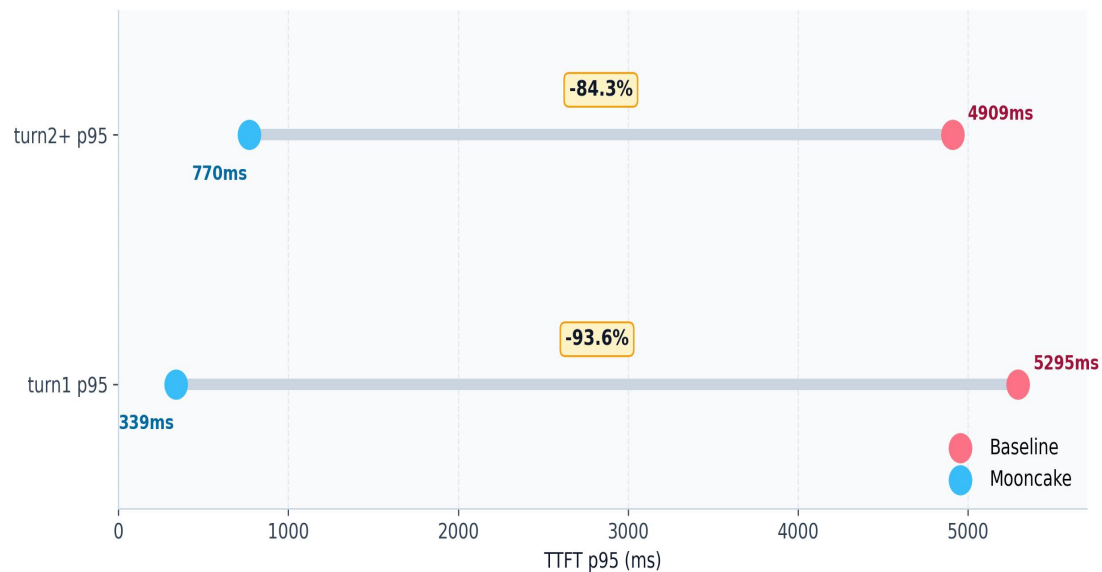
- turn1 p50: 756ms → 301ms (**2.5x**)
- turn1 p95: 5295ms → 339ms (**15.6x**)
- turn2+ p50: 171ms → 169ms (持平)
- turn2+ p95: 4909ms → 770ms (**6.4x**)

核心洞察

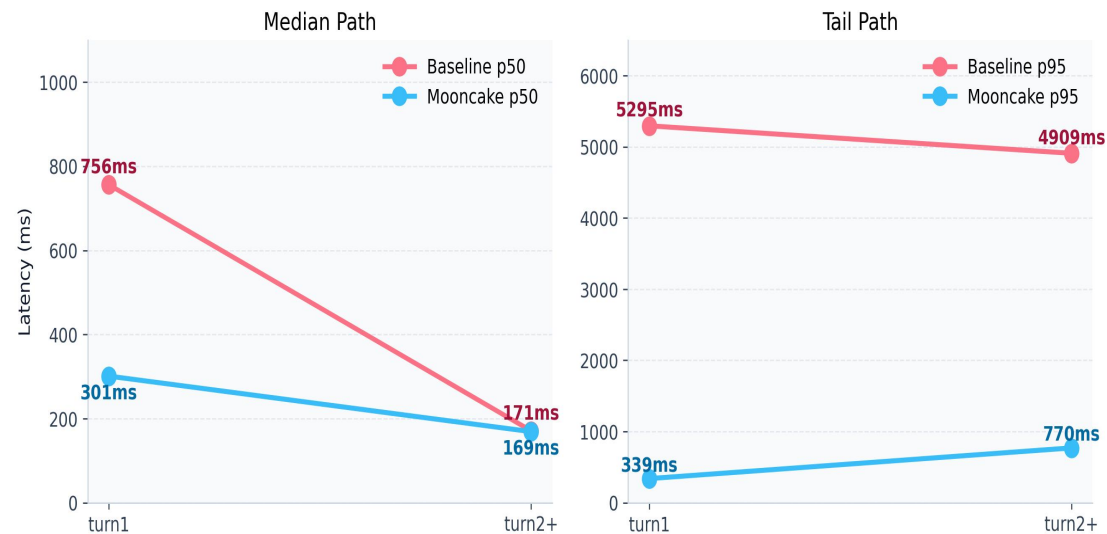
最大收益在尾部延迟，而非中位数

尾延迟压缩: Mooncake 核心价值

Mooncake Pulls Tail Latency Back Down



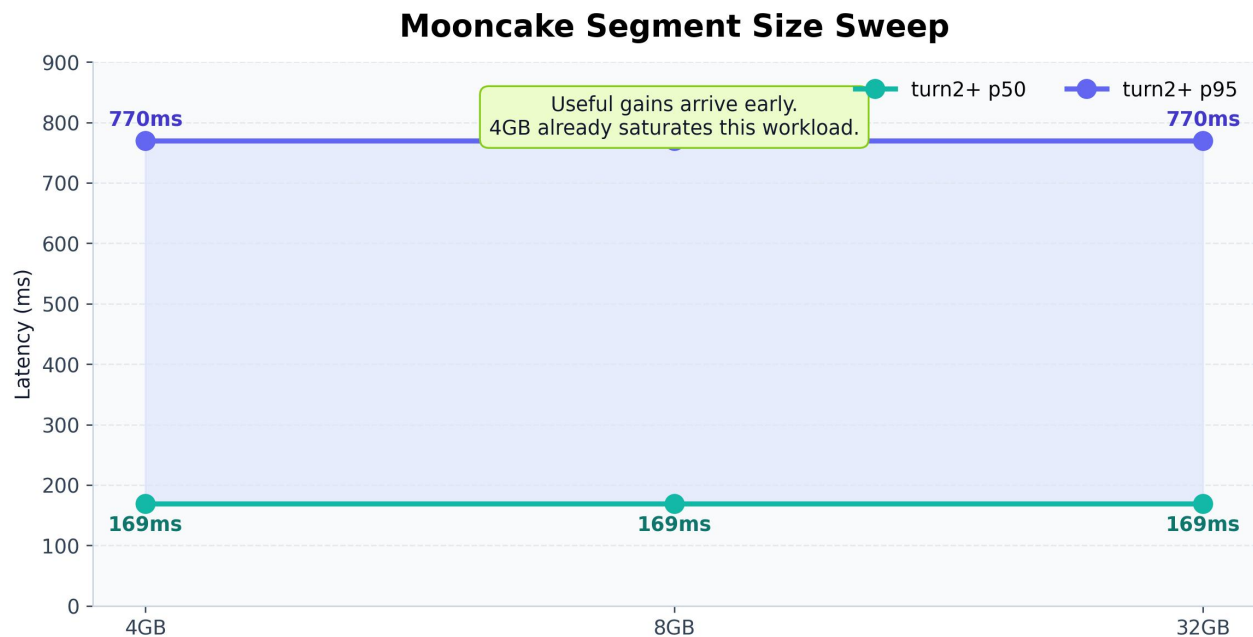
Mooncake Changes the Tail Much More Than the Median



KEY INSIGHT

Mooncake 不改变最快请求的速度, 而是改变最慢请求被允许的慢度。将系统从"通常快但偶尔很慢"变为"一致平滑"。

Segment Size 增益饱和分析



配置对比结果

- **4GB / 8GB / 32GB** 配置: 性能完全一致
- turn2+ p50: **169ms** (三个配置相同)
- turn2+ p95: **770ms** (三个配置相同)
- **关键发现** 增益在 4GB 即饱和

INSIGHT

Mooncake 的收益来得很早 — 工作集适配后，更大缓存不带来额外收益。这意味着成本效益极高。

06

多 Agent 记忆优化的关键技术

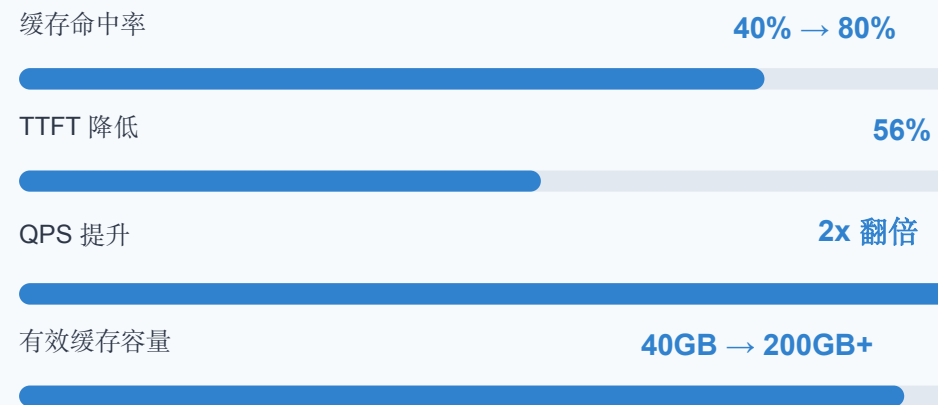
HiCache / Tokencake / KVzip / Helium / Ilm-d

缓存编排: HiCache 三级存储

GPU	HBM	热缓存, 即时计算
Host	DRAM	温缓存, 扩展容量
Remote	3FS	冷缓存, 持久化 + 跨实例共享

核心创新: **HiRadixTree** — 跨存储层级前缀匹配

生产数据 (Novita AI)



空间-时间调度: Tokencake

T 时间维度

Agent 等待工具返回 → GPU 空转

事件驱动 KV Cache 卸载 → CPU

预测模型估算工具时长, 提前回传 GPU

S 空间维度

多 Agent 竞争 KV Cache 容量

DAG 关键路径分析 (深度 x 出度)

共享池 + 保留池 动态划分

-47% 延迟降低

+16.9% 内存利用率

60ms KV 传输

vs 9000ms 重计算开销

KV Cache 压缩前沿

KVzip

NeurIPS 2025 Oral

- 查询无关 (Query-Agnostic) 压缩
- 压缩时确保支持任意未来 query
- 对 Agent 关键: 未来工具调用不可预测

EvicPress

统一压缩框架

- 统一有损压缩 + 自适应淘汰
- 根据内存压力动态决策: 压缩/迁移/淘汰
- 端到端自适应内存管理

SideQuest

Agent 专用架构

- 专为长周期 Agent 任务设计
- "共享上下文并行推理"架构
- 防止管理性 token 污染主上下文

工作流感知: Helium 全局优化

Helium (arXiv'26) — 把 Agent 工作流当"查询计划"优化

核心方法

- 把 Agent 工作流当"查询计划"优化
- TRT (Task Relation Tree) + 主动缓存
- Cache-Aware 调度策略
- 19-Agent, 88-LLM 算子端到端工作流

vs naive vLLM

39.5x

端到端加速

关键消融实验

移除 Plan Pruning	↓ 23.35%
移除 KV Cache 优化	↓ 3.55%

"看到全局结构" 比 "优化单点缓存"

重要 **6x** 以上

Memory-Aware 请求路由: llm-d

Red Hat / IBM — K8s 原生 KV Cache 感知调度

架构

- 1 vLLM Pod 发出 **KVEvents** 流
缓存状态变更实时上报
- 2 **全局索引** 追踪每个 Pod 持有的前缀
- 3 **Precise Prefix-Cache Scorer**
做出路由决策

8 vLLM Pods · 16 H100 GPUs

P90 TTFT 结果

vs 近似调度	57x
vs 随机路由	170x

唯一在生产级 K8s 做
KV Cache 感知调度的系统

Mooncake 赋能 Agent Memory: 端到端方案

Agent Memory 不应停留在向量 DB — 需要下沉到物理推理基础设施

Multi-Agent 协作 (A2A/MCP)

共享上下文 + 工具调用 + 状态传递



记忆感知调度 (Memory-Aware Routing)

llm-d 前缀索引 / Helium 工作流 DAG / Tokencake 空间-时间



Transfer Engine: 跨节点零拷贝 KV 传输

KVCache Store: 多级缓存 (VRAM→DRAM→3FS)

全局共享池: 一次计算, 全局复用

Mooncake: 物理记忆基础设施



硬件加速 (CXL 记忆池化 / RDMA / NVMe-of)

物理层高速互联, 支撑上层记忆基础设施

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO



07

协议与缓存的融合

当前最大的架构盲区与未来方向

协议层与缓存层的 Gap

协议层 (A2A / MCP) 缺失

MISSING

- 无缓存位置提示
- 无前缀对齐指令
- 无会话亲和性
- 无缓存生命周期提示
- 无传输元数据

影响量化

MCP 动态加载工具

命中率 85% → 0%

A2A Task 跨 Agent 移交

TTFT 慢 136x

JSON 序列化不确定

TTFT 慢 65%

追加大量工具返回

延迟反增 8.8%

生产实践：缓存感知的上层设计

CASE 1

Claude Code 四层上下文架构

- L1: 静态系统提示+工具定义 (永久)
- L2: 项目上下文 (每项目)
- L3: 会话上下文 (每会话)
- L4: 对话消息 (仅追加)

92% 前缀复用, 81% 成本降低

CASE 2

Manus 工具冻结

- 所有工具始终在 prompt 中
- Logits Masking 控制可用性
- 避免工具集变更导致缓存失效

稳定的前缀缓存命中

CASE 3

确定性序列化

- sort_keys=True
- 固定字段顺序
- 避免 JSON 不确定性导致缓存 miss

消除无意义缓存失效

不是算法创新, 而是理解物理约束并让上层设计与之对齐

弥合 Gap: 缓存语义融入协议

<code>session_id / workflow_id</code>	缓存亲和性路由	避免跨 Agent re-prefill
<code>prefix_hash</code>	基础设施级前缀匹配	无需解析 prompt 即可路由
<code>expected_tool_duration</code>	KV Cache TTL 估算	避免工具调用间缓存淘汰
<code>workflow_dag</code>	全局缓存预热	主动预加载下一步 Agent 的 KV
<code>shared_context_ref</code>	跨 Agent 上下文复用	KVCOMM 式偏移修正

ANALOGY

类比: HTTP 被迫承载了 Cookie / Cache-Control / ETag — A2A/MCP 将走同样的路

五大趋势判断

1

KVCache 从"内存管理"升级为"一等系统资源"—NVIDIA CMX 硬件专用

2

从前缀匹配到近似复用—KVCache 的向量检索

3

协议层增加缓存语义不可避免—类比 HTTP Cache-Control

4

Agentic Serving 独立化—专用 benchmark (TPC-C for Agent)

5

Agent Memory 下沉到物理基础设施—不应停留在向量 DB

Mooncake 社区路线图 & Open Problems

近期 Near-term

- 完善 3FS 层次化存储集成
- 强化 Multi-Agent 缓存调度
- CXL 内存池化原型

中期 Mid-term

- 协议层缓存语义标准化
- 跨数据中心 KV 迁移 (PrfaaS)
- Agent-native 推理 Benchmark

远期 Long-term

- Agent Memory OS
- 认知压缩引擎
- 统一记忆流转框架

动态控制流

ReAct 循环和运行时分支如何优化?

安全与隔离

多Agent共享KV Cache的信息泄露风险

缓存一致性

知识库更新时共享KV Cache如何失效?

统一成本模型

延迟/吞吐/成本/命中率的全局最优

标准化 Benchmark

Agentic推理的TPC-C在哪里?

THANKS

Q&A

开源地址: github.com/kvcache-ai/Mooncake
官网: kvcache.ai
马腾

AiCon

全球人工智能开发与应用大会